

06

Authorization

Who can see what, and where generated code leaves the check out

A missing authorization check produces no error and no visible symptom, so the app keeps working until somebody requests data belonging to another user. That makes it one of the few defects you can ship for months without noticing, and it is among the more common serious defects in AI-assisted code. This chapter gives you the model that explains where checks belong, the nine shapes the failure takes, and one thirty-second test that settles most doubts with evidence.

Assumes the trust boundary from chapter 1: anyone can send your app's requests by hand, with the parts they don't want removed.

The model

Two separate questions

Authentication and authorization are separate questions, often handled as one under the heading of “permissions.”

Authentication asks *who are you?* It’s resolved once, at login, and produces an identity.

Authorization asks *are you allowed to do this specific thing to this specific row?* It’s resolved on every operation.

Login can work perfectly while authorization is entirely absent. The confusion worth naming is that “there is a logged-in user” is a different claim from “this row belongs to them” — and code that checks the first while skipping the second is the most common authorization defect in generated code, since it’s shorter to write and there’s a visible check for a reviewer to land on.

The four places a check can live

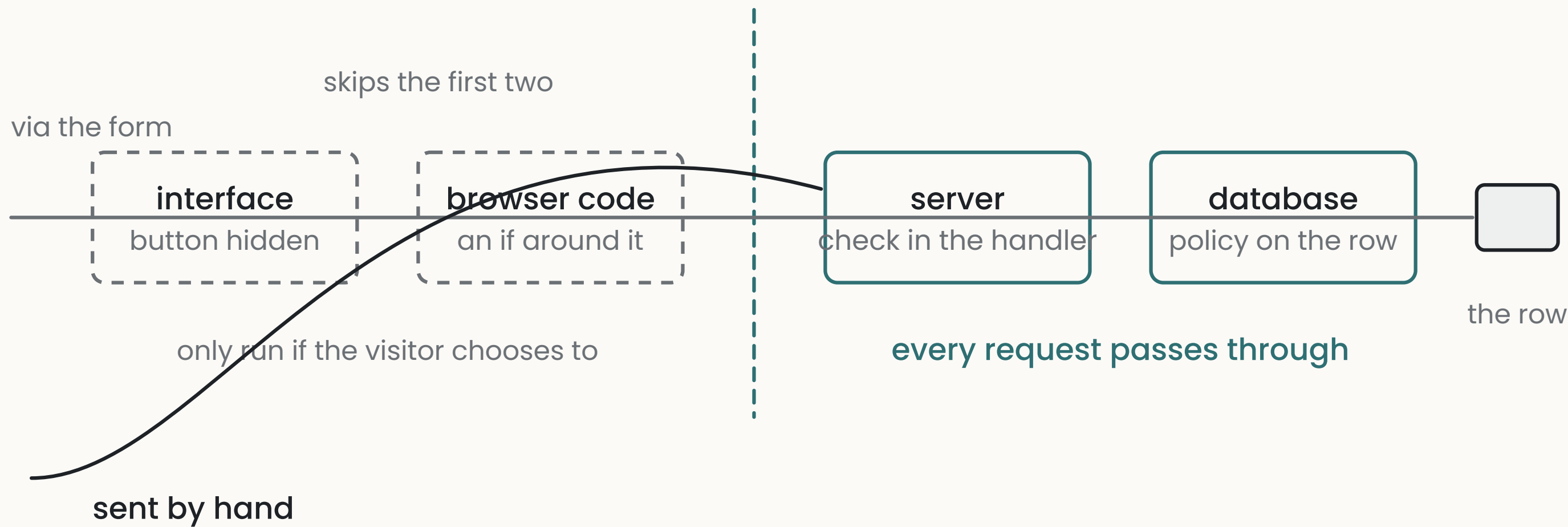
When your app decides whether someone may do something, that decision can be enforced in four places.

WHERE	REAL?	WHAT IT DOES
The interface — the button is hidden	fake	Helps usability, since it stops honest users hitting errors
Browser code — an <code>if</code> around the action	fake	Nothing, because the attacker doesn’t run your code
The server — a check in the request handler	real	Covers the requests that go through this handler
The database — a policy on the row	real	Covers every request that reaches the data

06

Authorization

THE FOUR PLACES A CHECK CAN LIVE



The first two are worth having, since an interface that shows buttons which always fail is confusing to use. They just don't survive a request that skips the interface, and skipping it takes one line in a browser console.

The gap between the last two is narrower and matters more. A check in a request handler covers that handler, so it won't apply to a second one added next month that touches the same table, or to a background job, or to an endpoint your assistant adds while you skim the summary. A policy in the database covers paths that don't exist yet, which is why it's usually the one to want first.

When your database has no policies. Row-level policies are a feature of some databases and the default convention on platforms built around them; this chapter’s examples use Supabase. On a stack where the server talks to the database with one shared credential — most frameworks with their own data layer — the database never learns who the user is, and the server check is the real one. The model still holds: the check belongs in the layer every path passes through. There, that means one place — a scoped query helper, a data-access module, a middleware — that every route and every job goes through, so that a route added next month inherits it. The failing shape is the same as here: a check written into one handler and absent from the next. Databases without row policies at all — MySQL, most document stores — put the whole chapter on that server layer, and shape 3’s ask becomes “for each table, what enforces ownership on each of the four operations.”

What a database policy is

A row-level security policy is a filter the database itself attaches to every query that touches a table — a condition like *the row’s owner must be the person asking*. The caller can’t remove it,

because it never appears in anything they send; the database adds it on arrival, whatever the request looked like.

Three things about policies are worth knowing even at this altitude:

1. **Turning protection on and writing the rules are separate steps.** A table with row-level security enabled and no policies denies everything — which breaks the app before it protects it, and is the moment many projects turn protection back off. A table with it disabled, which is the default on most platforms, accepts everything.
2. **Permissions are per-operation.** Reading, creating, updating and deleting are separate grants, so a table with only a read policy still accepts writes from anyone. A protected table has all four accounted for.
3. **There is a master key.** Platforms issue a privileged credential (Supabase calls it `service_role`) that bypasses every policy by design. It exists for trusted server-side work. It must never reach the browser, and it is never the fix for a permissions error — the policy that raised the error is the thing to look at.

One more piece: the identity a policy checks against comes from the login system's signed token, so the caller can't supply it. A `user_id` arriving in the request body is just text the caller typed, and any code treating it as identity has handed the decision to the attacker.

Authentication, briefly

This chapter leaves the login itself to the platform: password handling, token signing, and the session mechanics are the part generated code least often gets wrong, since the platform's library does them. What surrounds the login is yours, and it fails the way authorization does — silently.

The **session token** is what the browser holds after login and sends with every request; it's the proof the policies check against. It has an expiry, after which the user is asked to log in again, and a logout is supposed to invalidate it. On some setups logout only forgets the token on the browser's side, and a copy taken earlier keeps working until it expires.

The login form is one front door, and most apps have several: password reset, magic links, invitation acceptance, email change, sign-in with another provider. Each ends in the same thing — a session for an account — and each has its own proof

that the person controls that account: a link only the account's email received, a code, a provider's signed assertion. Generated code builds these doors one at a time, and the weaker ones are where the shape in entry 6 recurs, with the account to act on taken from the request rather than from the proof.

Why generated code gets this wrong

The failure isn't random; it has causes, which is why it recurs.

- The interface conditional is what “add permissions to this” looks like in the code already on screen, so that's where the assistant adds it.
- The database policy lives in a different system — a migration file or a dashboard — usually outside the files the assistant was asked to change.
- Client-side checks are shorter, and they read as complete.
- A request to “make it work” can be satisfied by removing the check that blocks it, and sometimes is.

None of this improves with a more capable assistant alone, since each cause is about where the work is visible rather than about capability.

It improves with asking, which is what the rest of this chapter is for.

PART

What goes wrong

There are nine shapes, and each carries the *tell* if you do look at code, the **ask** to put to your assistant, and the **check** that settles it with evidence.

1 The UI-only gate

the button is hidden and the endpoint is open.

ASK “For each action in this feature, state every place it’s enforced: interface, browser code, server, database policy. Flag any action whose only enforcement is in the first two.”

CHECK send the request the hidden button would have sent, from a browser console (chapter O, D2), and see whether it succeeds.

TELL *the only mention of a role or permission sits in interface code.*

2 Authenticated but not authorized

the session is confirmed and ownership never is.

ASK “For every read and write of this table, state the ownership condition. Say ‘none’ where there isn’t one.”

CHECK the direct test below.

TELL *a logged-in check followed by a query whose only filter is an id.*

3 New table, protection off

a migration adds a table, which the API then exposes with nothing on it.

ASK “List every table with its row-level security status and its policies for read, insert, update, delete. Flag any table with fewer than four operations covered.”

CHECK request the table with the public key, logged out. Zero rows is the pass.

TELL *a create table with no accompanying enable-row-level-security.*

5 Partial coverage

reads are restricted and writes are open, or updates are checked on the way in and nothing examines where the row lands.

ASK the enumeration from shape 3 — the same list answers both.

CHECK as a normal user, attempt the operation the list says is uncovered.

TELL *fewer than four operations addressed on a table users can write to.*

4 The master key used to clear an error

a permissions error blocks progress and the privileged credential makes it go away.

ASK “List every place a privileged key is used, and for each, why the request can’t work with the public key.”

CHECK search the built client bundle for the key’s value (chapter O’s D3 prompt builds it and searches). If it was ever in the browser, rotate it — whether anyone found it is a separate question from whether the door was open.

TELL *a service or admin credential in a file the browser can reach, or a commit that fixes a permissions error without touching a policy.*

6 Identity taken from the request

a user id arrives in the payload and becomes the ownership field.

ASK “For each write endpoint, state where the user’s identity comes from: the session token, or the request body.”

CHECK send the request with someone else’s id in the body and see whose name ends up on the row.

TELL *a userId pulled from a request body.*

7 The unprotected neighbour

the target table is protected and a lookup table it trusts isn't, so access can be read sideways or granted by writing to the neighbour.

ASK “For each policy that consults another table, state that table’s own policies. Flag any that are permissive.”

CHECK as an unauthorized user, try to write the row in the neighbour table that would grant you access.

TELL *a policy whose correctness depends on another table’s contents, with no check of that table’s own policies.*

8 The public bucket

the rows are protected and the files they point at are served to anyone with the link.

ASK “For each place files are stored, state whether the bucket is public or private, and for private ones, what a request for a file must carry to be served. Say public explicitly.”

CHECK copy a file’s URL from a logged-in session and open it in a private window.

TELL *uploads stored in a bucket marked public, or file URLs that work in a private window.*

9 The second front door

a recovery or invitation path that establishes a session with weaker proof than login.

ASK “List every path by which a session can be created or an account’s email changed — password login, magic link, reset, invitation, provider sign-in, email change. For each, state what proves the person controls the account, and where the account being acted on comes from. Say nothing where nothing proves it.”

CHECK request a reset for your own test account, then use the link with a different account’s id or email substituted.

TELL *a reset, email-change, or invite handler that reads the target account from the request body.*

THE DIRECT TEST

Sign in as user B. Request one of user A’s rows by its id. An error or an empty result is the pass; the row itself is a finding.

This one test settles shapes 2, 3 and 5 for the table you run it on, produces evidence rather than an inference, and takes under a minute once set up. Your assistant can build you the setup: ask it for a way to run requests as a specific test user against your project, and to show you the exact request to send. Run it on the table whose exposure would matter most — the one holding other people’s private data — before any other.

If you find nothing, record why, specifically. “Every table has row-level security on with four policies each, and cross-user reads fail on the three tables holding user data” is checkable later, in a way that “it looked fine” isn’t.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like, since the failure mode of asking is an answer that reassures without informing. If you get prose instead of the artifact, re-ask insisting on the list.

06

Authorization

D1 · Audit my project

AUDIT

List every table in my project. For each one: is row-level security on, and which of read, insert, update, delete have policies? Present it as a table and flag any row with fewer than four operations covered. Then, for every endpoint that writes data, state where the user’s identity comes from: the session token or the request body. Don’t summarize — I want the full list, with ‘none’ stated explicitly wherever something is missing.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is two tables you could tick off row by row. Follow up on every flag with: “for this one, show me the request that would exploit it.”

D3 · Walk the failure path

WALK THE FAILURE

Walk me through, step by step, what happens when someone who isn’t logged in sends a request directly to my backend for all rows of [your most sensitive table] — including any API my database platform generates automatically, not just routes I wrote myself. Start from the request arriving. At each step, say what could stop the request there, and whether in my project anything actually does.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response reaches a definite ending: blocked at a named step, or rows returned. Ask for the request itself so you can run it and watch which ending is true.

D2 · Explain my own app to me

GROUNDING EXPLANATION

Here is my schema: [paste it, or point at the project]. For each table, tell me in plain language: who can read it and who can write it, today, as deployed — and what enforces that in each case: a database policy, server code, or nothing. Where the honest answer is “anyone” or “nothing,” say exactly that.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response names an enforcement mechanism per table. “The app only shows users their own data” is describing the interface, which is shape 1 — ask where that’s enforced.

D4 · Quiz me

PREDICTION QUIZ

Quiz me on authorization in my own app. One scenario at a time: describe a request someone could make, ask me what I think happens, wait for my answer, then tell me what actually happens and why. Cover at least: an anonymous read, a logged-in user reading someone else’s row, a write with someone else’s id in the body, and one scenario I wouldn’t think to ask about.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit to each answer before reading the correction — a wrong prediction points at the specific part of your picture that needs fixing, which reading rarely surfaces on its own.

D5 · Set the house rules

BUILD A TOY

Put this in the instructions file your assistant reads (`CLAUDE.md` , `AGENTS.md` , `.cursor/rules` , or your tool’s equivalent — chapter O’s D4 names it), then re-run a recent feature request in a fresh session and compare what comes back against what you originally merged:

- Every new table gets row-level security enabled in the same migration that creates it, plus explicit policies for read, insert, update, delete.
- The user’s identity always comes from the session token, never from a request body, query parameter, or client-supplied field.
- The privileged key is server-only and is never used to resolve a permissions error. If a query is blocked by a policy, examine the policy.
- Interface conditionals on roles are usability only, never sole protection.
- Files live in private buckets and are served only to callers the owning row’s policy would admit.
- When adding a data-access path, state in your summary which policy protects it. If none does, say so explicitly rather than proceeding.

WHAT A GOOD RESPONSE LOOKS LIKE

The last rule turns an omission into a sentence you can notice while reading a summary, which is easier than inferring it from what the summary doesn’t contain.

06

Authorization

WHERE THIS CONNECTS

Chapter 1 · Client and server is the model this chapter stands on. **Chapter 7 · Secrets and configuration** picks up the master key — shape 4 is where the two chapters meet. **Chapter 8 · Injection** covers what a request’s text can make your side do once it’s past the check. **Chapter 12 · Verification** generalizes the ask-and-check method used here to every surface.

The full-depth version of this chapter exists as [a five-sitting lab manual](#) — you build a small multi-user project, break it, protect it, and write real policies against a checker that passes or fails. Worth doing if your project holds other people’s data; the manual assumes no programming experience.